Step 1: Cluster Orchestration with Amazon ECS

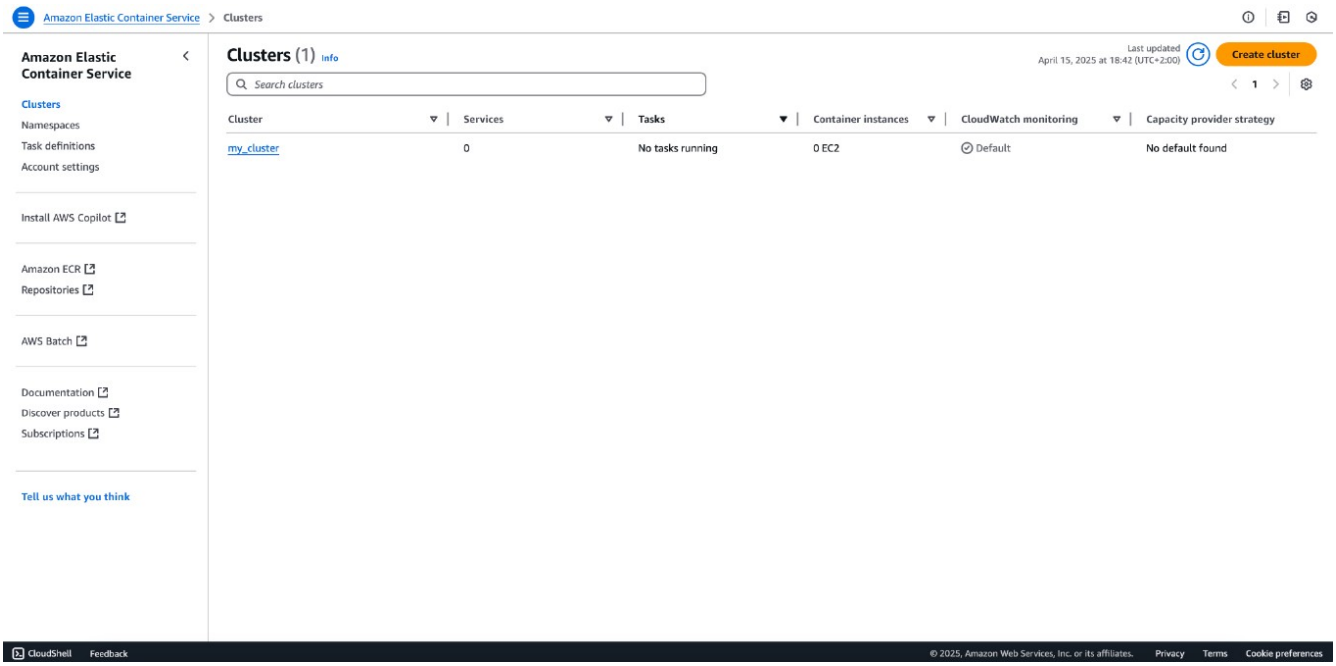
Technical Description

In this initial phase, I provisioned an **Amazon Elastic Container Service (ECS)** cluster named `my_cluster`. This serves as the foundational logical grouping of resources required to run containerized applications.

Key Takeaways & Learning Points

To make this entry more valuable for a **Cybersecurity/DevSecOps** portfolio, I've highlighted the core architectural decisions made during this step:

- **Infrastructure Strategy:** The cluster is configured to support both **AWS Fargate** (serverless) and **Amazon EC2** capacity providers. For this lab, using Fargate is preferred from a security standpoint as it abstracts the underlying server management, reducing the attack surface and removing the need for manual OS patching.
- **Control Plane vs. Data Plane:** Understanding that ECS acts as the *Control Plane* (managing where and how containers run), while the instances or Fargate tasks act as the *Data Plane* (where the code actually executes).
- **Security Observation:** At this stage, the cluster is an empty shell. The next critical security steps involve defining **IAM Roles** (Task Execution Role) and **Security Groups** to ensure the principle of least privilege is applied before any traffic reaches the containers.



Step 2: Secure Image Management with Amazon ECR

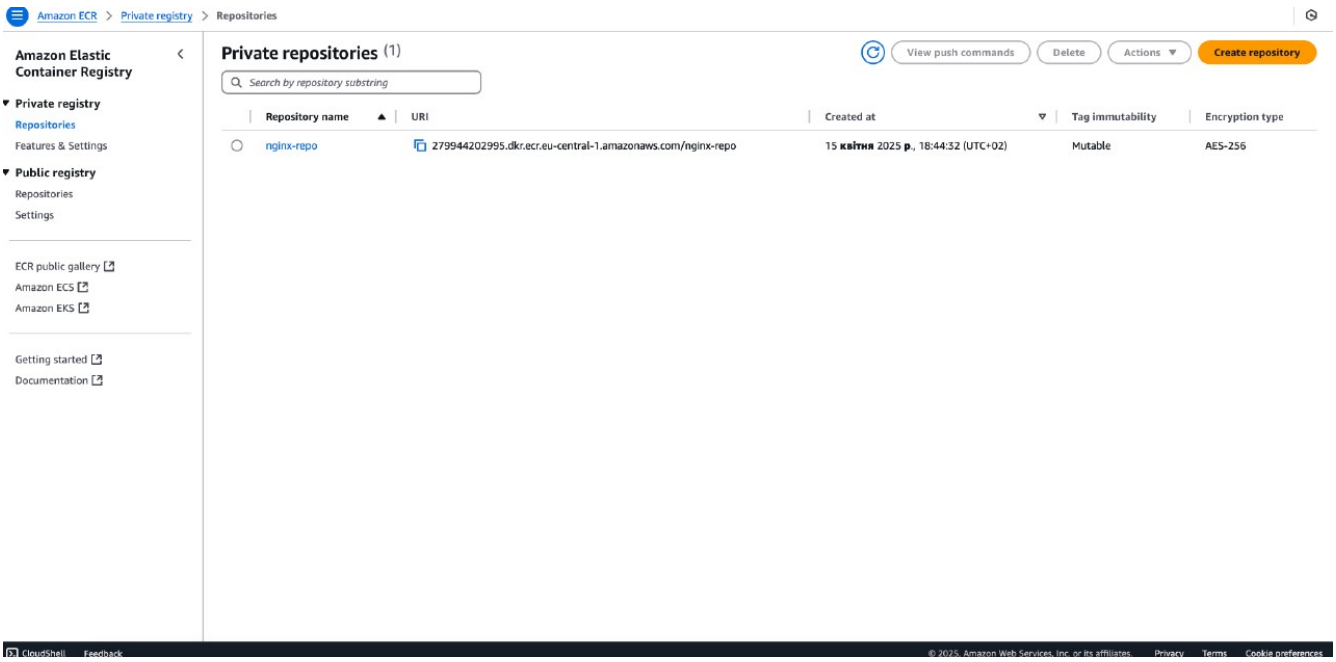
Technical Description

In this step, I established a **Private Repository** within **Amazon Elastic Container Registry (ECR)** named `nginx-repo`. This repository acts as a secure, scalable, and managed Docker registry,

allowing for the storage and management of container images that will be deployed to the ECS cluster created in the previous step.

Security & Educational Highlights

- **Encryption at Rest:** The repository is configured with **AES-256 encryption**, ensuring that container images are encrypted while stored in the AWS cloud.
- **Tag Immutability:** While currently set to "Mutable," in a high-security production environment, I would enable **Tag Immutability**. This prevents existing images from being overwritten by new ones with the same tag (e.g., `latest`), protecting against accidental or malicious changes to deployed software versions.
- **Vulnerability Scanning:** Integrating Amazon ECR with "Scan on Push" allows for the automatic detection of software vulnerabilities within the container images immediately upon upload—a core practice in a **Shift-Left security** approach.
- **Private vs. Public Access:** By utilizing a **Private Registry**, the application's core logic and base images remain inaccessible to the public internet, accessible only via authenticated IAM identities or specific VPC endpoints.



Step 3: Container Image Preparation and Deployment to ECR

Technical Description

This step involved the local preparation of the container image and its subsequent upload to the AWS cloud. I utilized the **Docker CLI** to pull the base image, retag it for compatibility with my specific AWS infrastructure, and securely push it to the previously created ECR repository.

Execution Workflow

The terminal commands executed (as shown in the screenshot) follow a standard industry workflow:

1. `docker pull nginx`: Retrieving the official Nginx image from Docker Hub.
2. `docker tag nginx [AWS_Account_ID].dkr.ecr.[Region].amazonaws.com/nginx-repo:latest`: Re-mapping the local image to the

ECR repository URI. This is a critical step for ensuring the image is correctly addressed within the AWS ecosystem.

3. `docker push ...`: Authenticating and uploading the image layers to the private registry.

DevSecOps & Cybersecurity Insights

- **Supply Chain Integrity**: By pulling an official image and immediately pushing it to a **Private ECR**, I am creating a "Golden Image" repository. This reduces reliance on external public registries, which can be subject to rate-limiting or malicious image poisoning.
- **Layered Security**: The terminal output shows multiple layers being "Pushed." In a DevSecOps pipeline, each of these layers can be analyzed for vulnerabilities. Using the `digest (SHA256)` visible in the output ensures **content addressability**, meaning I can verify that the image running in production is exactly the same as the one I pushed, with no unauthorized tampering.
- **Identity and Access Management (IAM)**: Successfully running the `push` command confirms that the local environment (or CI/CD runner) is properly authenticated using **AWS CLI credentials** with the specific `ecr:PutImage` and `ecr:BatchCheckLayerAvailability` permissions, adhering to the principle of least privilege.

```
> docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:09369da6b10306312cd908661320086bf87fbae1b6b0c49a1f50ba531fef2eab
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
> docker tag nginx 279944202995.dkr.ecr.eu-central-1.amazonaws.com/nginx-repo:latest
> docker push 279944202995.dkr.ecr.eu-central-1.amazonaws.com/nginx-repo:latest
The push refers to repository [279944202995.dkr.ecr.eu-central-1.amazonaws.com/nginx-repo]
455e93166cde: Pushed
3e0d80cccd09: Pushed
79e69c90ece3: Pushed
054c7ade68e5: Pushed
3094e3f45803: Pushed
77e72e56d2dc: Pushed
c9b18059ed42: Pushed
latest: digest: sha256:03bdbdce7b7ca3b728c33ea5fee7d51cd9b89bb951082ccf106065d40abfd2ed size: 1778
```

Step 4: Creating a Blueprint (Task Definition)

Technical Description

In this step, I created a **Task Definition** named `nginx-task`. Think of a Task Definition as a "blueprint" or a set of instructions that tells AWS exactly how to run our container. Without this blueprint, the cloud wouldn't know which image to use or how much power the application needs.

What’s inside this blueprint?

Looking at the screenshot, we configured the following key settings:

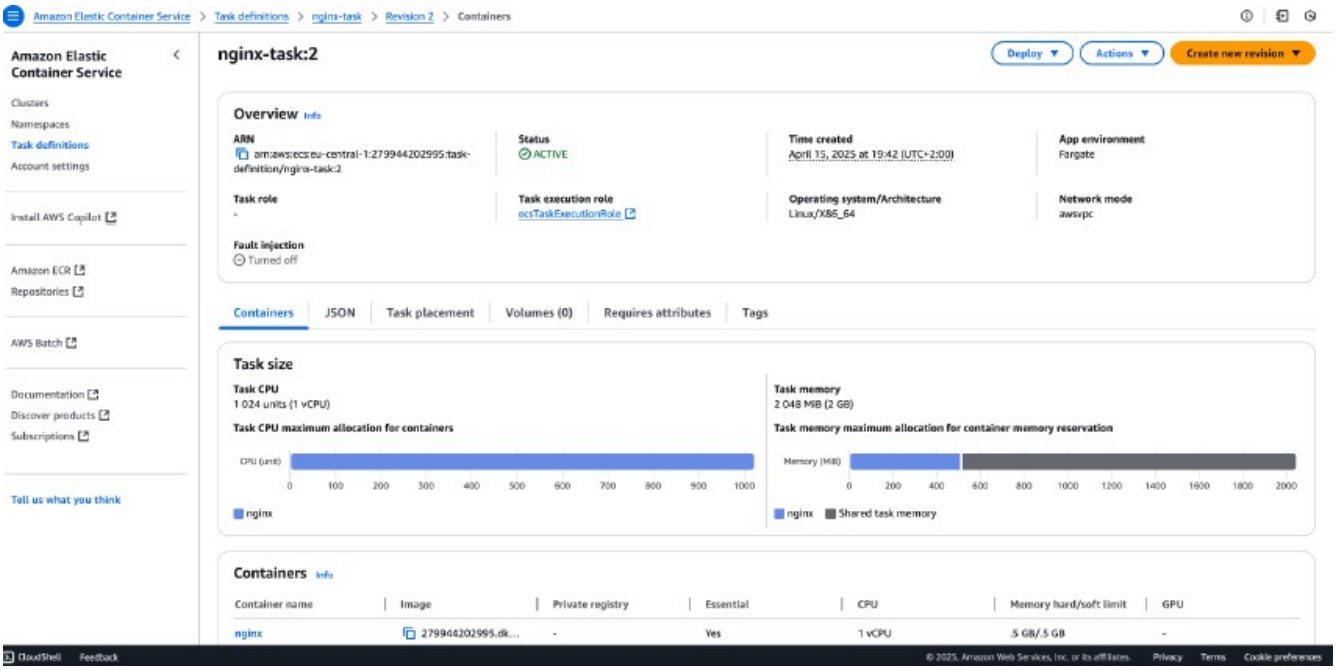
- **Infrastructure (Launch Type):** I chose **Fargate**. This means AWS manages the servers for us, so we don't have to worry about maintaining the underlying hardware.
- **Resources:** I allocated **1 vCPU** and **512 MB of Memory**. This is enough "brain power" for a simple web server like Nginx to run smoothly.

- **Container Image:** I linked it to the `nginx-repo` image we uploaded to **ECR** in the previous step. This tells AWS exactly what software to run.
- **Connectivity (Port Mapping):** I opened **Port 80**. This is the standard "door" for web traffic, allowing people to view the website.

Why this matters for Security

From a security perspective, this step is where we define the "boundaries" for our application:

1. **Limited Resources:** By setting a specific CPU and Memory limit, we ensure that if something goes wrong, the container won't consume all our cloud resources.
2. **Specific Roles:** I assigned an `ecsTaskExecutionRole`. This is like a specialized ID card that only gives the container the exact permissions it needs to download the image from ECR and nothing more.
3. **Isolation:** Since we are using Fargate, each task runs in its own isolated environment, which is a great way to keep our application safe from other processes.



Step 5: Launching the Service (Making it Live)

What are we doing?

Now that we have our "blueprint" (Task Definition) from the previous step, it's time to actually run it. In this step, I created an **ECS Service** named `nginx-ser`.

Why are we doing this?

Running a single container is easy, but in the professional world, we need reliability. The Service ensures that our application stays running. I configured the **Desired Tasks** to **2**. This means AWS will always try to keep two copies of my website running at the same time. If one crashes, the Service automatically starts a new one to replace it.

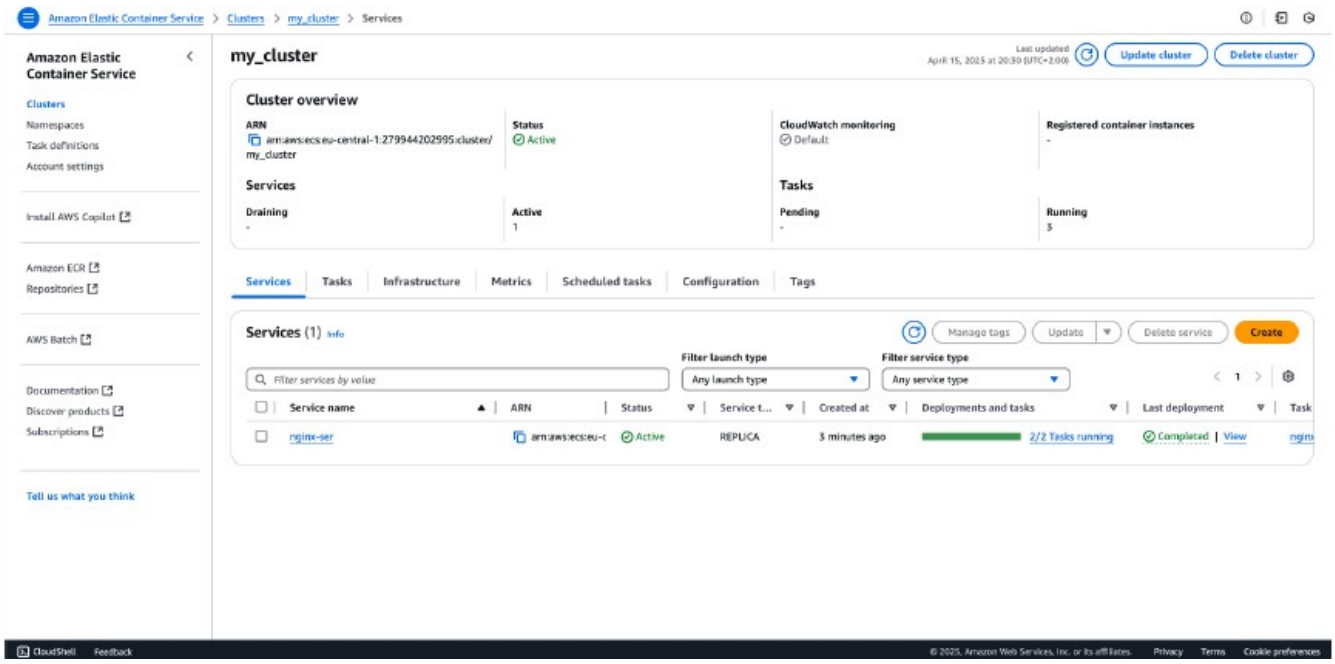
Key Settings in this Step:

- **Launch Type (Fargate):** Again, we use Fargate so we don't have to manage any physical or virtual servers.

- **Deployment Type (Replica):** This tells AWS to maintain a specific number of identical copies of our application across the cluster.
- **Status (Active/Running):** As seen in the screenshot, the service is "Active," and the green bar shows that both tasks are successfully up and running.

Security & Reliability Notes

- **High Availability:** By running **2 tasks** instead of just one, we improve security through availability. If one instance is attacked or fails, the other continues to serve users, preventing a "Denial of Service" (DoS) situation.
- **Automatic Recovery:** The service monitors the health of the containers. If a container becomes "unhealthy" or stops responding, the service kills it and launches a fresh, clean instance from our secure ECR image.
- **Isolated Networking:** The service is running within a private network structure, ensuring that only the traffic we specifically allowed in our security groups can reach these containers.



Step 6: Setting up the Application Load Balancer (ALB)

What are we doing?

In this step, I created an **Application Load Balancer** named `nginx-alb-service`.

Why are we doing this?

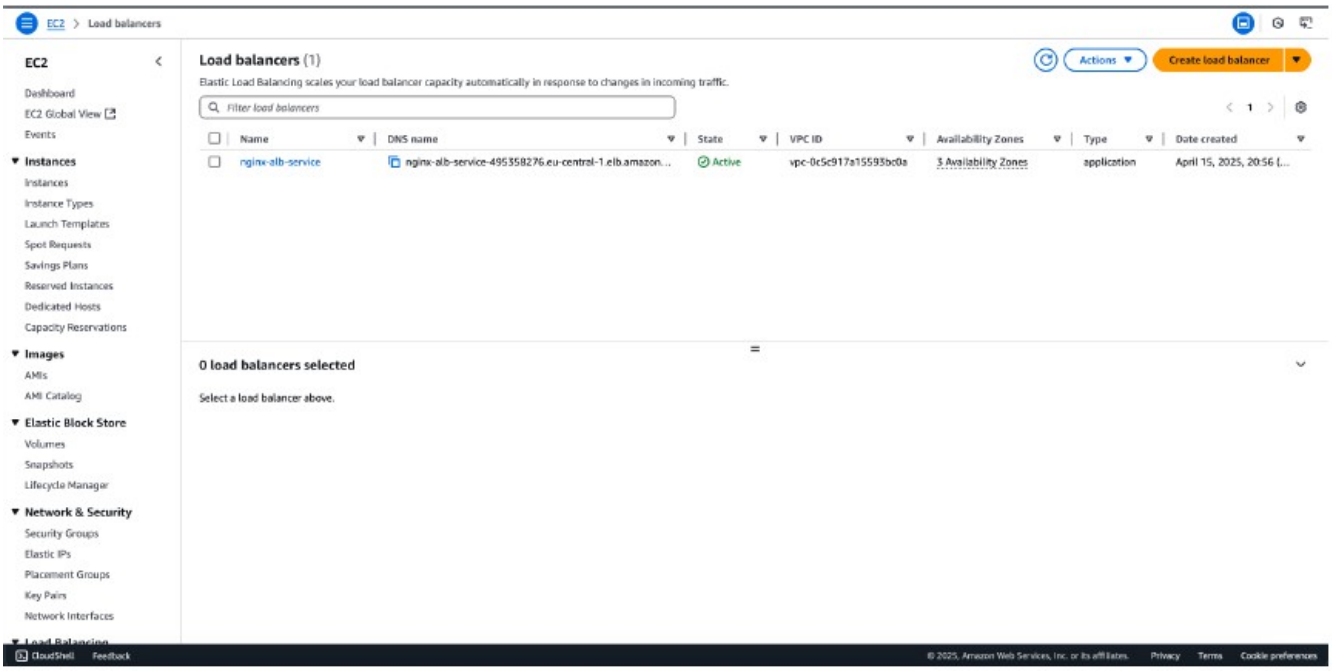
We don't want users to connect directly to our containers for several reasons:

- **Traffic Management:** Since we have 2 tasks running (from Step 5), the ALB balances the traffic between them. If one container is busy, the ALB sends the next user to the second container.
- **A Single Point of Entry:** Users only need one web address (DNS name) to reach our site, no matter how many containers we add or remove behind the scenes.

- **Status Check:** As seen in the screenshot, the status is **"Active"**, meaning the balancer is ready and currently monitoring the health of our application.

Security & Performance Notes

- **Hiding Infrastructure:** The ALB acts as a shield. Users never see the private IP addresses of our containers; they only interact with the Load Balancer. This makes it much harder for attackers to target our actual servers.
- **Health Checks:** I configured the ALB to perform "Health Checks." If one of our containers starts acting up or becomes compromised, the ALB will immediately stop sending traffic to it and wait for a healthy one to take its place.
- **DDoS Protection:** By using an AWS Load Balancer, we automatically get a layer of protection against common network attacks (like DDoS), as AWS infrastructure handles the heavy lifting of managing thousands of incoming connections.



Step 7: Configuring the Target Group (The Connection)

What are we doing?

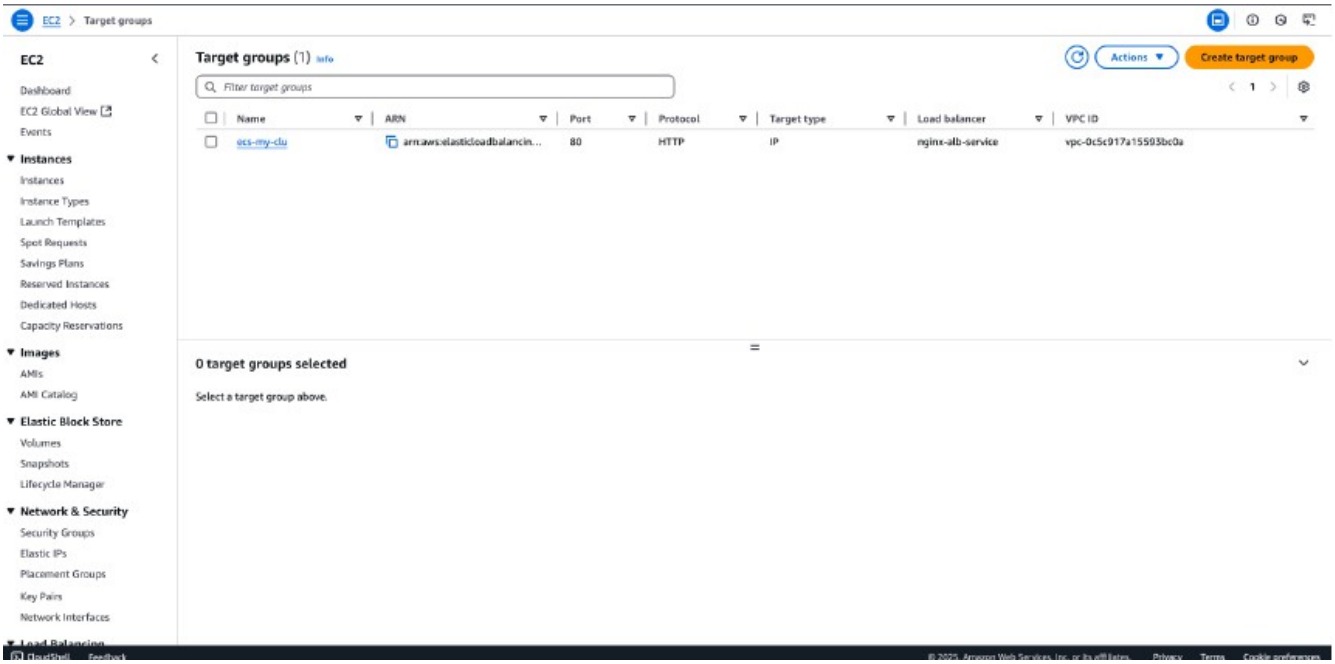
In this step, I configured a **Target Group** named `ecs-my-clu`.

Why are we doing this?

The Load Balancer needs to know exactly which resources are ready to handle traffic.

- **Directing Traffic:** The Target Group tells the ALB: "Send all incoming web traffic on Port 80 directly to the IP addresses of my ECS tasks."
- **Logical Grouping:** It allows us to treat a group of independent containers as a single entity. Even if we scale up to 10 containers, the Target Group keeps track of them all so the Load Balancer doesn't get confused.

- **Health Monitoring:** As seen in the screenshot, the Target Group is linked to the `nginx-alb-service`. It continuously checks if the containers are "healthy" before sending any users their way.

Security & Performance Notes

- **Tightened Access:** By using a Target Group, we ensure that the containers only accept traffic that comes *specifically* from our Load Balancer. This prevents someone from trying to bypass our security "front door" and attacking the containers directly.
- **Protocol Security:** We are using the **HTTP protocol on Port 80** for this lab. In a real-world cybersecurity scenario, the Target Group would often be configured to work with HTTPS (Port 443) to ensure that all data traveling between the user and our application is encrypted.
- **Zero Downtime:** Because the Target Group knows the status of every container, it enables "Green/Blue" deployments. We can start new containers, wait for the Target Group to verify they are healthy, and only then start sending users to the new version.